

Графовые базы данных: Neo4j

§0. Введение в графовые базы данных: Neo4j



Общая характеристика Neo4j

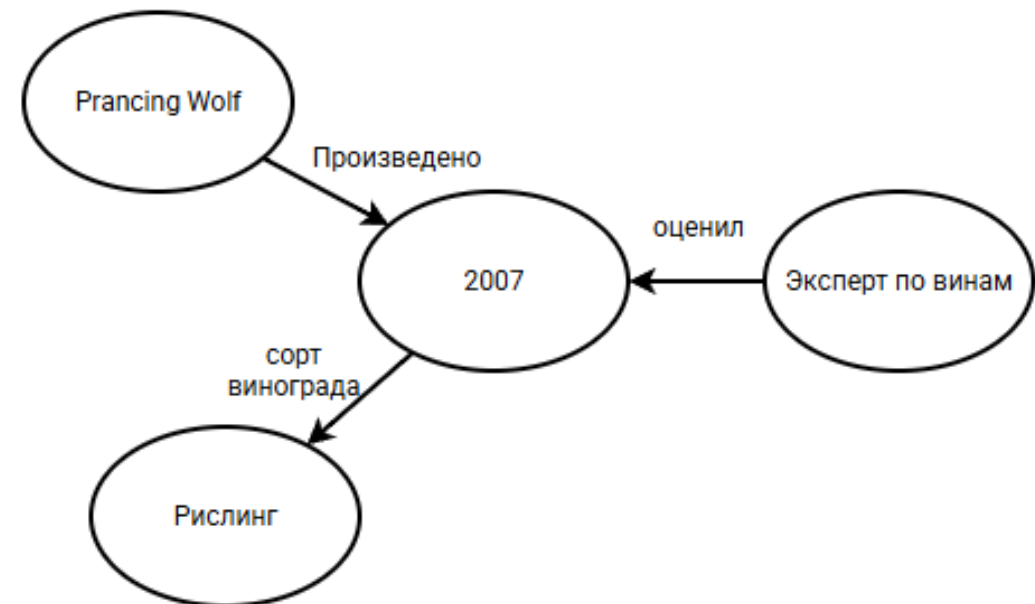
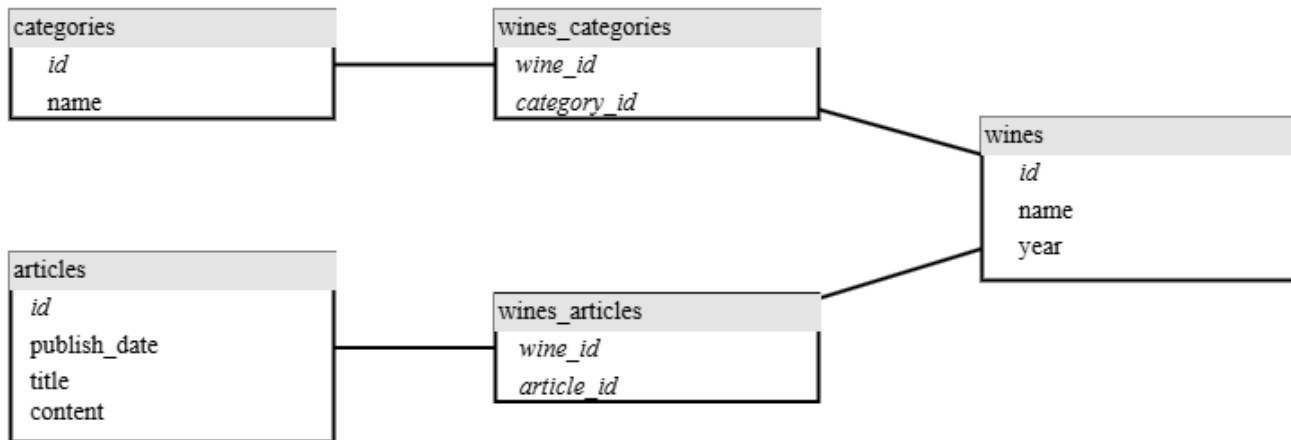
- Neo4j – новый тип хранилищ данных NoSQL, получивший название графовая база данных.
- Данные в ней хранятся в виде графа.

Характеристика

- Упор делается на связях между значениями, чем на общие характеристики значений.
- Можно хранить десятки миллиардов узлов и столько же ребер.
- Поддержка репликации главный-подчиненный на много серверов.

Пример отличия Neo4j от других СУБД

- Необходимо создать систему рекомендаций вин, в которой для вина могут быть определены сорт, регион производства, виноградник, винтаж и десигнация.
- Также может потребоваться хранить статьи авторов, описывающих вина. Или дать возможность пользователям пометать свои любимые вина.



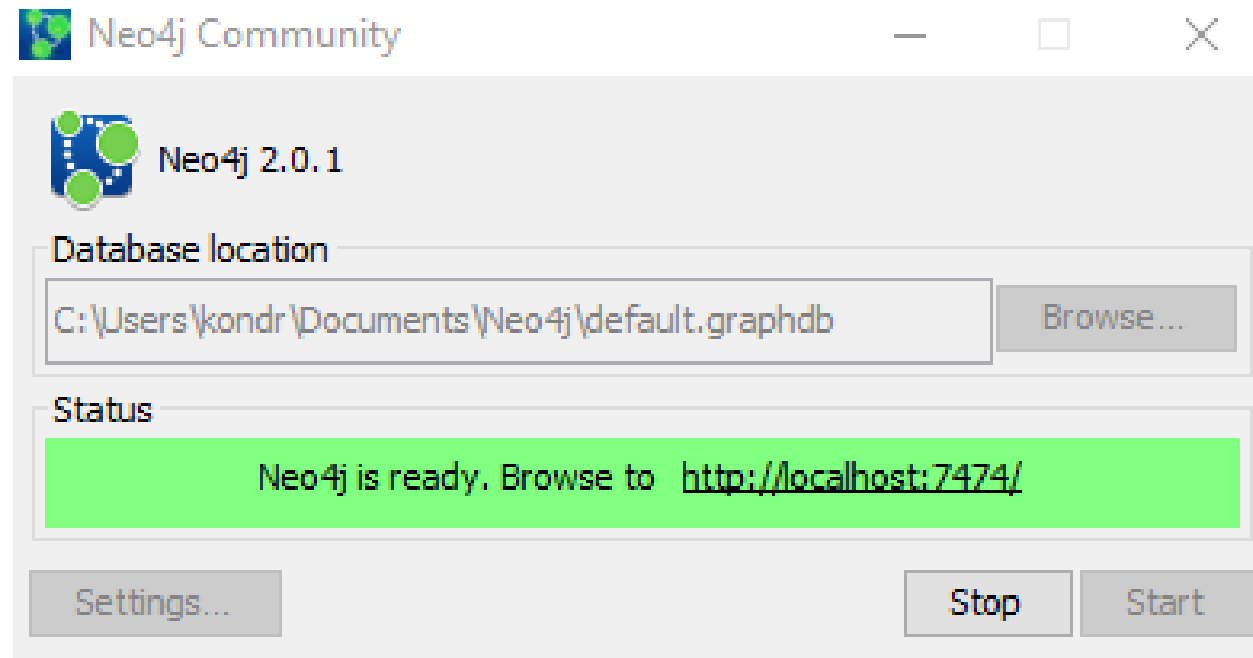
Графовые базы данных: Neo4j

*§1. Начало работы с веб-интерфейсом.
Создание элементов графа: наполнение ГБД*



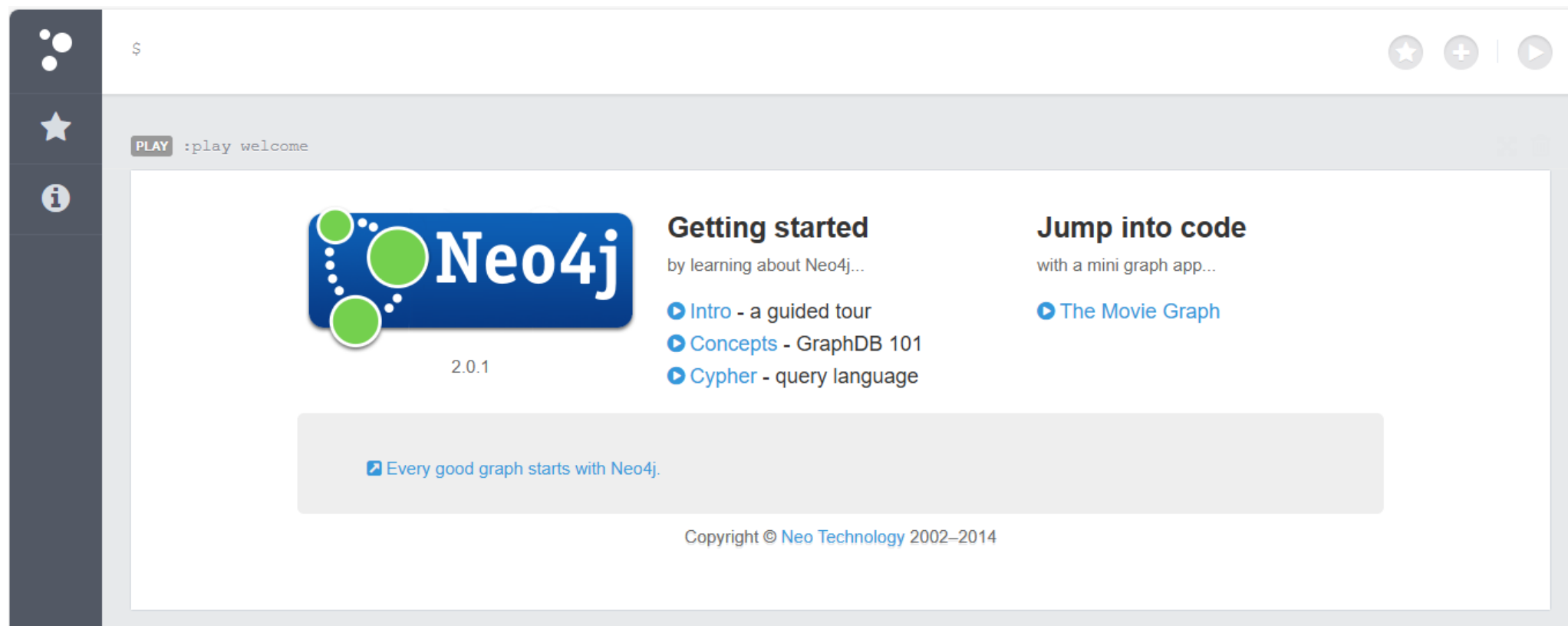
Первый запуск веб-интерфейса

- После удачного запуска сервера можете нажать на ссылку <http://localhost:7474/> или скопировать и вставить ее в любой другой доступный браузер.



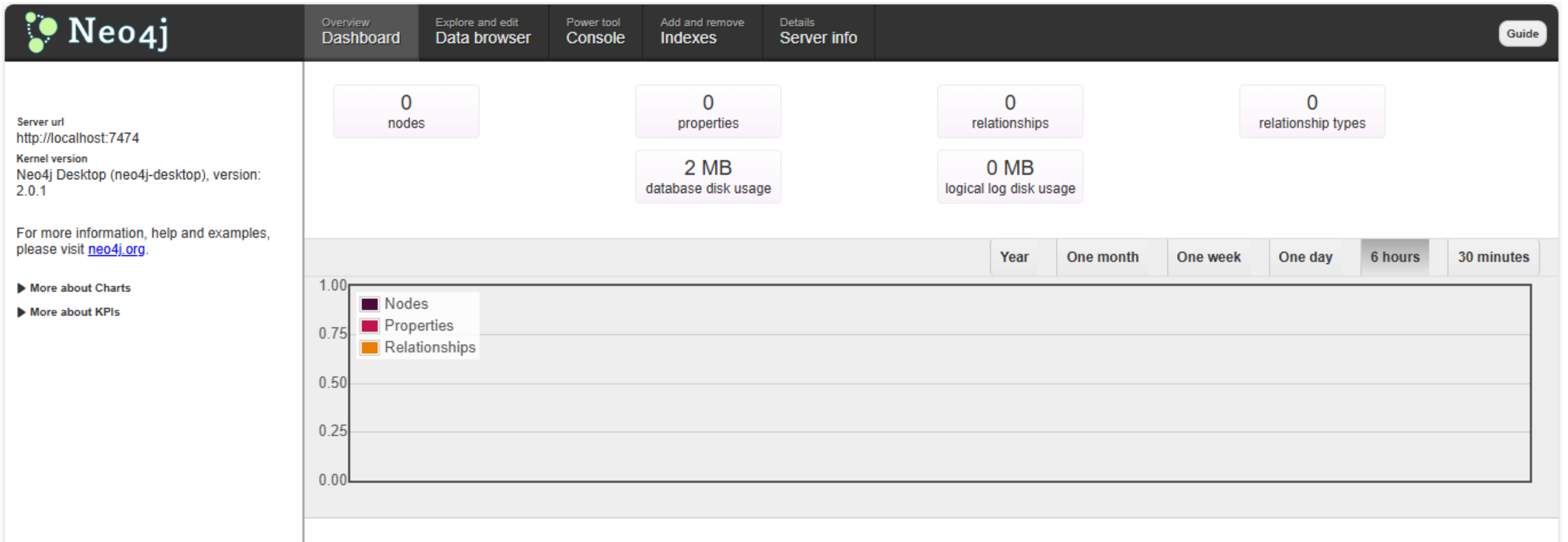
Веб-интерфейс Neo4j

- Стандартный пакет Neo4j включает административный веб-интерфейс и средства просмотра данных.



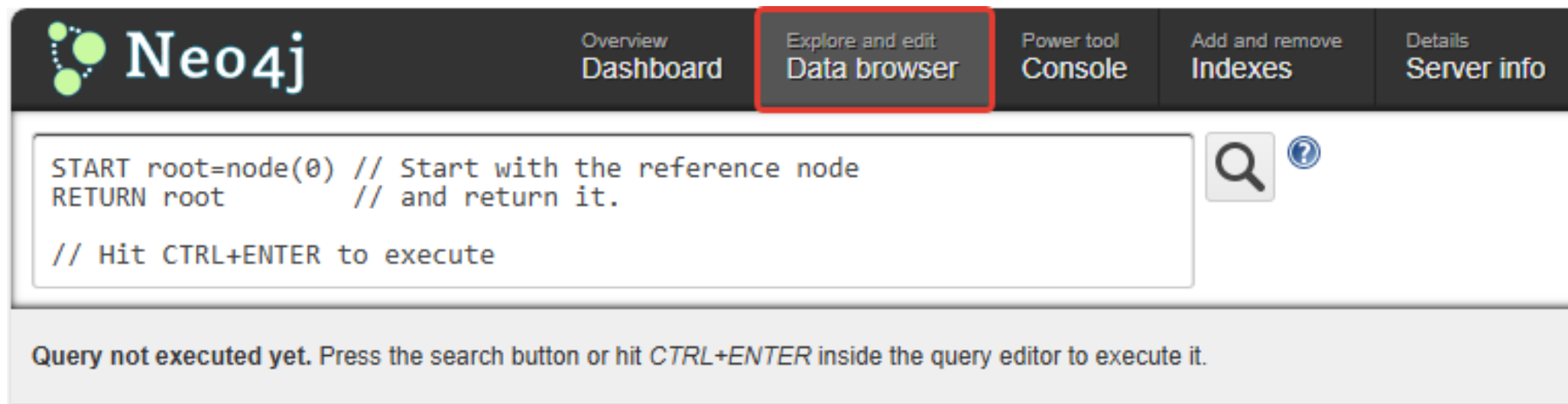
Переход на страницу администрирования

- Перейдите на страницу администрирования: <http://localhost:7474/webadmin/>.



Переход в обозреватель данных

- Перейдите в раздел – Data browser.
- В Neo4j под узлом понимается то же самое: вершина, из которой исходят ребра, где могут храниться данные в виде пар ключ-значение.



Добавление первого узла

- Нажмите на кнопку +Node и задайте ключ name и значение «Prancing Wolf Ice Wine 2007» (представляющие конкретное вино и винтаж), нажав на кнопку +Add property.

The screenshot displays the Neo4j Data browser interface. The top navigation bar includes the Neo4j logo and several tabs: Overview Dashboard, Explore and edit Data browser (highlighted with a red box), Power tool Console, Add and remove Indexes, Details Server info, and a Guide button. Below the navigation bar is a search bar containing the character 'ø' and a search icon. To the right of the search bar are two buttons: '+ Node' (highlighted with a red box) and '+ Relationship', followed by a graph icon. The main content area shows the results of a query: 'Returned 1 row. Query took 60ms'. Below this, a section for 'Node 0' is displayed, with the URL 'http://localhost:7474/db/data/node/0'. To the right of the node information are three buttons: 'Show relationships', 'Saved', and 'Delete'. Below the node information is a form with a label 'name' and a text input field containing the value 'Prancing Wolf Ice Wine 2007'. To the right of the input field is a 'Remove' button. At the bottom left of the form is a '+ Add property' button.

Добавление второго узла

- Для вновь созданного узла добавьте свойство name со значением Wine Expert Monthly.
- Номер узла автоматически увеличится.

Node 1
<http://localhost:7474/db/data/node/1>

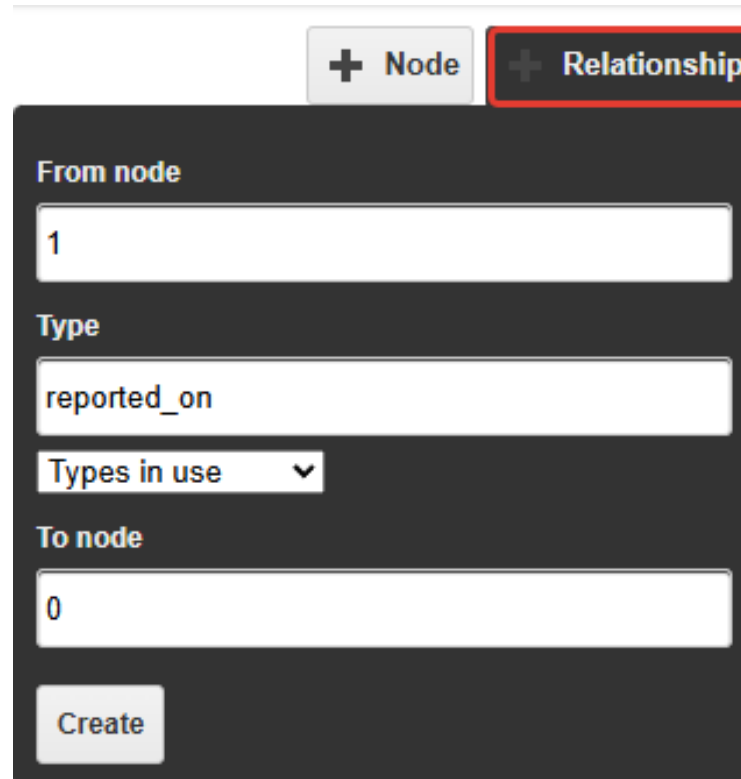
name

"Wine Expert Monthly"

+ Add property

Установка первой связи

- Поскольку журнал Wine Expert Monthly оценивал вино Prancing Wolf, то необходимо связать эти узлы, создав ребро.
- Нажмите кнопку +Relationship и установите связь, идущую из узла 1 в узел 0, присвоив ей тип reported_on.



The screenshot shows a web interface for creating a relationship in a graph database. At the top, there are two buttons: "+ Node" and "+ Relationship". The "+ Relationship" button is highlighted with a red border. Below these buttons is a form with the following fields:

- From node:** A text input field containing the number "1".
- Type:** A text input field containing the text "reported_on".
- Types in use:** A dropdown menu showing "Types in use" with a downward arrow.
- To node:** A text input field containing the number "0".
- Create:** A button at the bottom of the form.

Добавление свойства для первой СВЯЗИ

- Как и у узлов, у связей могут быть свойства.
- Нажмите кнопку +Add property и введите свойство [rating: 92] – это оценка, выставленная вину.

Relationship 0
<http://localhost:7474/db/data/relationship/0>

Node 1 reported_on Node 0

rating	92
--------	----

+ Add property

Добавление третьего узла

- Рислинг (сорт винограда) – это общая категория, которая может быть ассоциирована и с другими винами.
- Создайте новый узел и задайте для него свойство [name: «riesling»]

Node 2
<http://localhost:7474/db/data/node/2>

name	"riesling"
------	------------

+ Add property

Установка второй связи

- Добавьте еще одну связь, идущую из узла 0 в узел 2, присвоив ей тип `grape_type`.
- Задайте свойство `[style: «ice wine»]`.

+ Node

+ Relationship

From node

0

Type

grape_type

Types in use ▼

To node

2

Create

Relationship 1

<http://localhost:7474/db/data/relationship/1>

Node 0

grape_type

Node 2

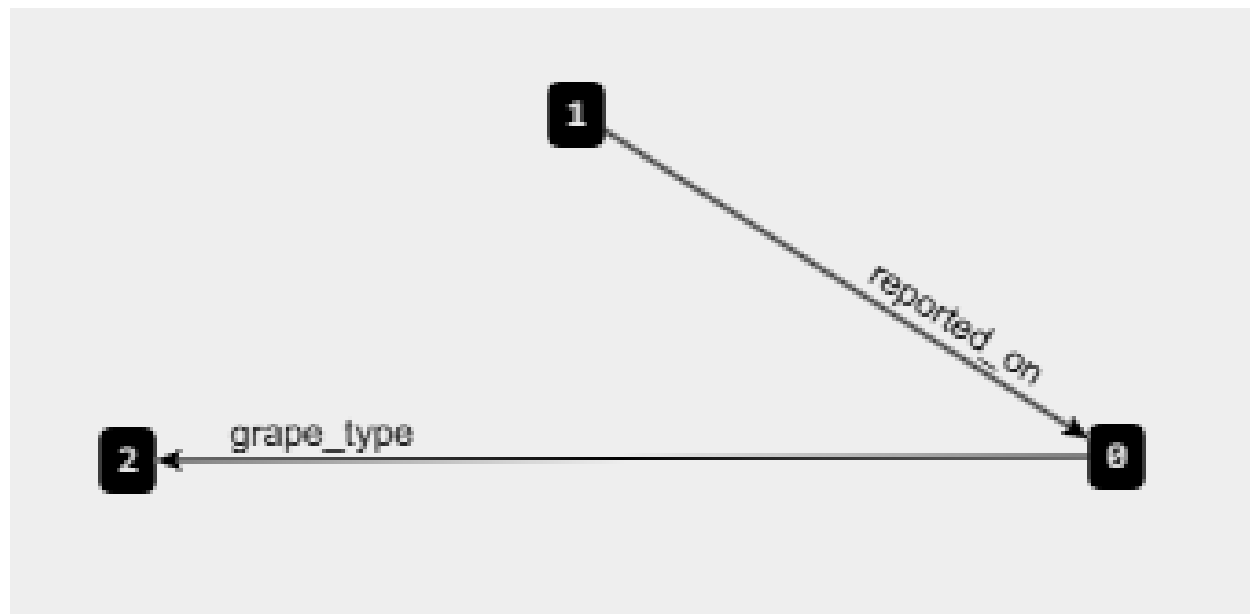
style

"ice wine"

+ Add property

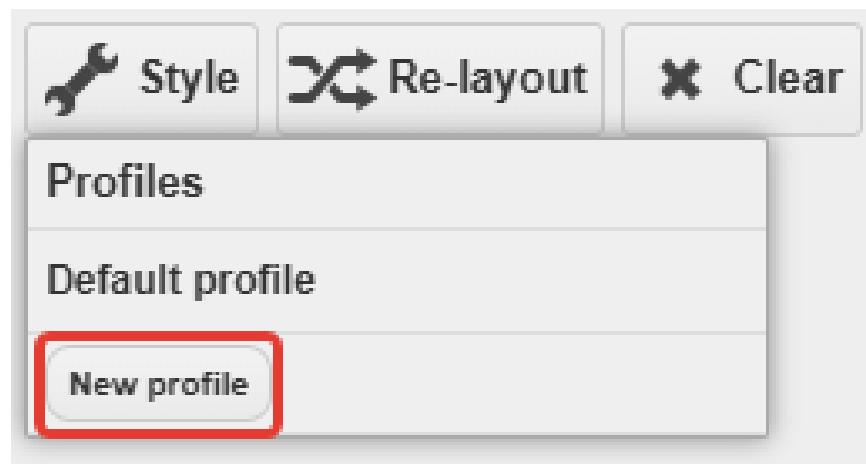
Просмотр графа

- Нажмите на кнопку  (рядом с +Relationship) и произойдет переключение в режим просмотра.



Изменение стиля графа

- Нажмите Style и выберите пункт меню New profile.



Создание специального профиля

- Введите в поле сверху имя wines, после чего измените метку с {id} на {id}: {prop.name}.
- После нажатия кнопки Save вы окажетесь снова на странице визуализации.

Create new visualization profile
wines

Style rules
Each style rule controls how either all nodes, or a filtered subset of nodes, should look in the visualization. The first matching style rule will be used, you can re-order the rules with the drag-handle under the each rules' Remove button.

Nodes
Remove
☰

Add filter

Show as
Box ▾
Background
[black box]
Label color
[white box]

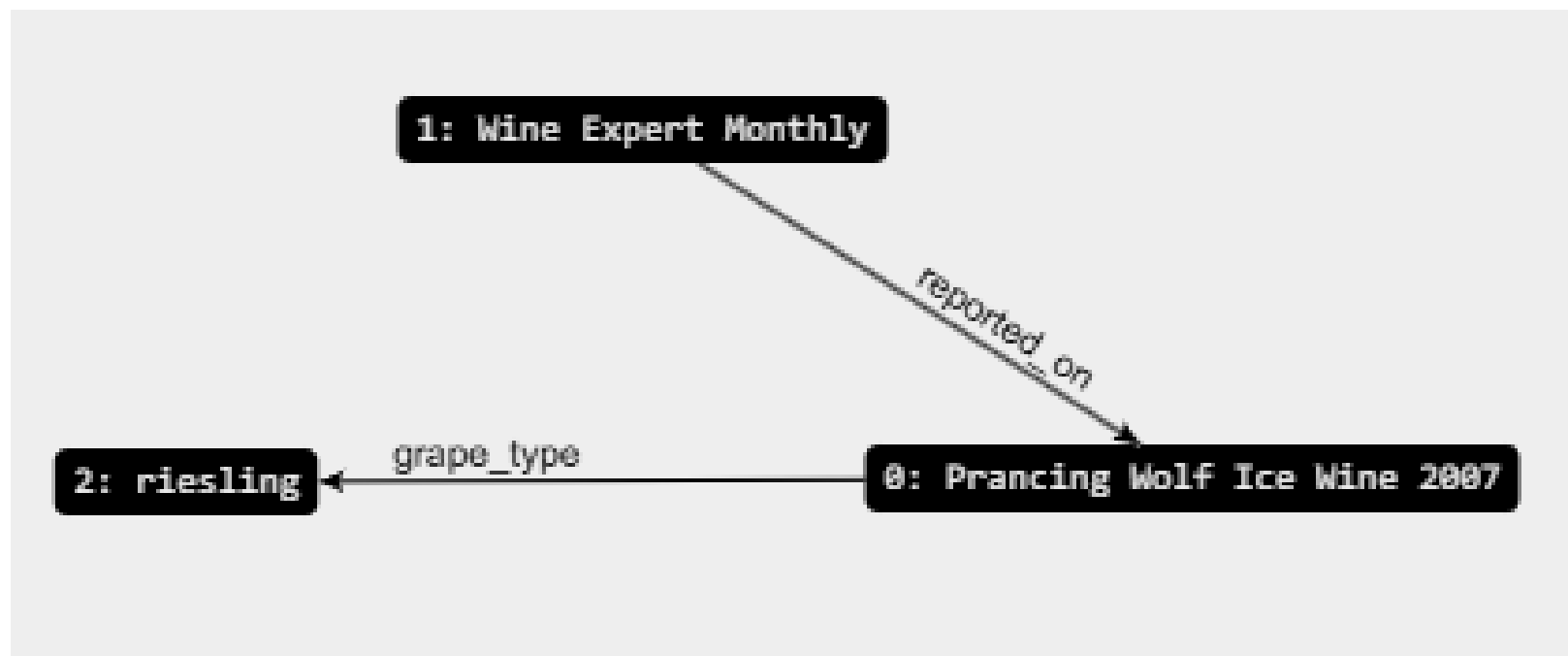
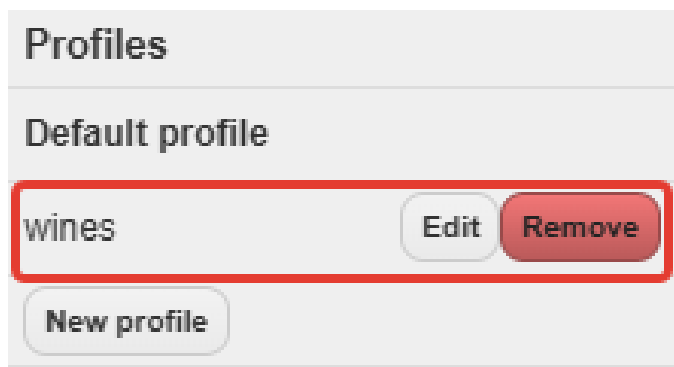
Label ⓘ
{id}: {prop.name}
Font size
10

Add style rule

Save Cancel

Визуализация графа узлов по специальному профилю

- Теперь из меню Style можно выбрать пункт wines.



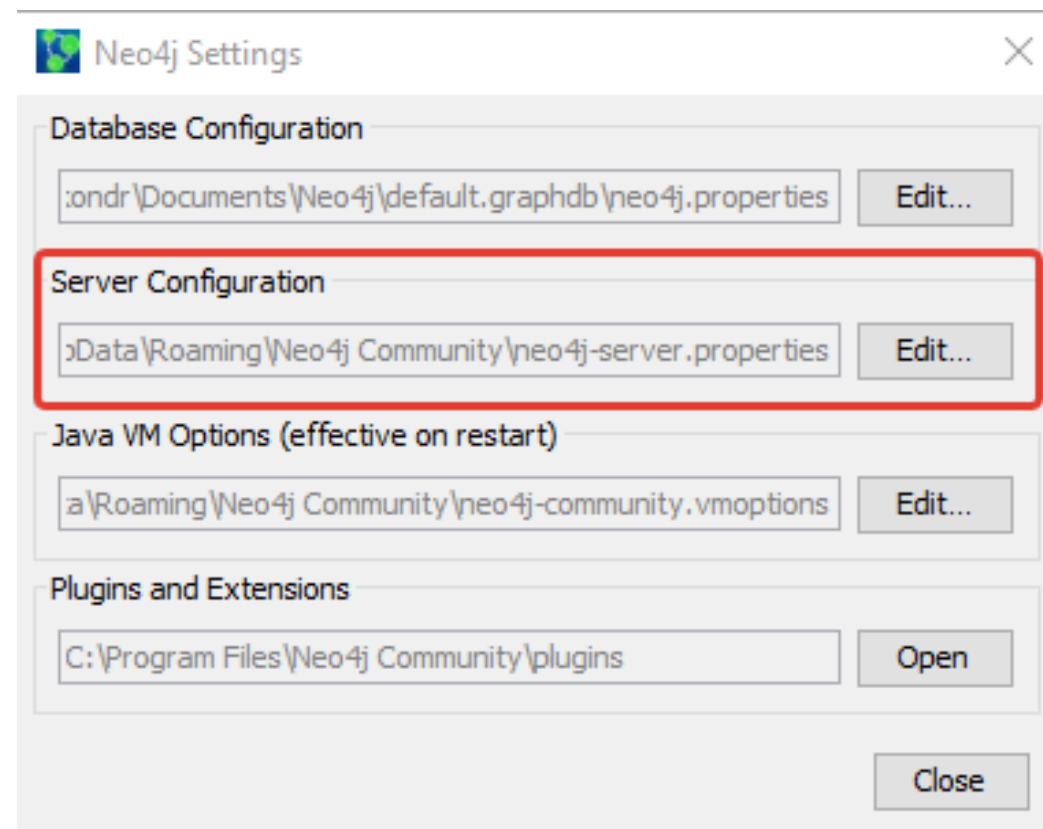
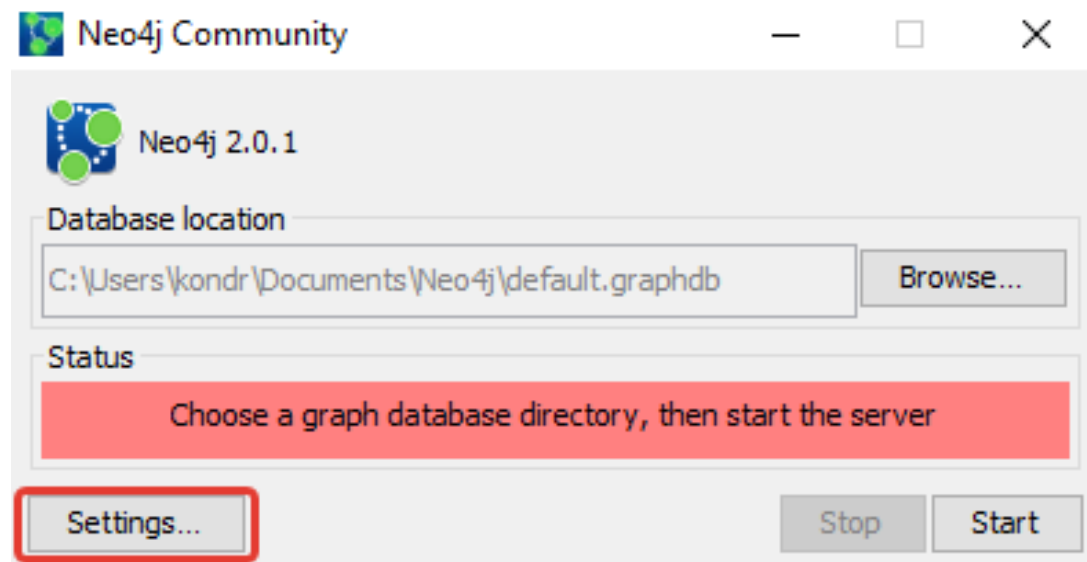
Графовые базы данных: Neo4j

§2. Настройка работы в Gremlin



Изменение конфигурации сервера

- При запуске Neo4j нажмите на кнопку Settings и в открывшемся окне перейдите в редактирование файла конфигурации сервера.



Изменение конфигурации сервера

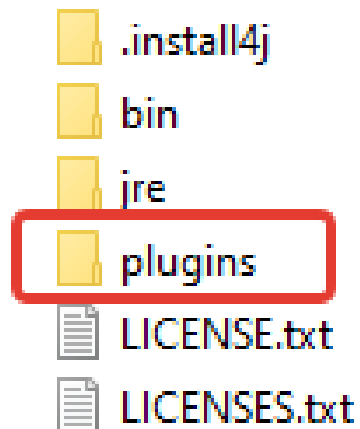
- Добавьте в конец текстового файла следующую строку:

```
# ubiquitous common log format  
#org.neo4j.server.http.log.config=neo4j-http-logging.xml  
org.neo4j.server.thirdparty_jaxrs_classes=com.thinkaurelius.neo4j.plugins=/tp
```

- Сохраните изменения.

Развертывание программного модуля

- Разархивируйте содержимое neo4j-gremlin-plugin-tp2-2.1.6-server-plugin в папку plugin, расположенный в корне Neo4j Community.
- Запустите сервер.



Проведение теста с использованием cURL

- Введите следующий запрос:
- При успешном выполнении отобразится пустое сообщение об успешном завершении при доступе к интерфейсу для взаимодействия с графовой базой данных с использованием протокола REST и языка запросов Gremlin.

```
C:\WINDOWS\system32>curl -s -G http://localhost:7474/tp/gremlin/execute | jq
{
  "success": true
}
```

- -s – "silent" mode (не отображает индикатор выполнения).
- -G – Использовать HTTP GET-запрос.

Общая характеристика Gremlin

- К Neo4j можно обращаться из нескольких языков: Java, REST-интерфейс, Cypher, Ruby и др.
- Gremlin предназначен для обхода графов.

Характеристика

- Gremlin дает доступ к инфраструктуре языка, на котором основан; в программе на Gremlin можно использовать конструкции на Groovy и библиотеки, написанные на Java.
- Переменная *g* представляет граф как объект.
- Действия над графом – это функции, вызываемые от имени данного объекта.
- В нем используются термины из математической теории графов.

Neo4j	Gremlin
Узел (node)	Вершина (vertex)
Связь (relationship)	Ребро (edge)

Графовые базы данных: Neo4j

§3. Работа в Gremlin



Доступ к множеству всех вершин графа

- Для доступа к множеству всех вершин графа имеется свойство с именем v.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.V" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "name": "Prancing Wolf Ice Wine 2007",
      "_id": 0,
      "_type": "vertex"
    },
    {
      "name": "Wine Expert Monthly",
      "_id": 1,
      "_type": "vertex"
    },
    {
      "name": "riesling",
      "_id": 2,
      "_type": "vertex"
    },
    {
      "_id": 3,
      "_type": "vertex"
    }
  ],
  "success": true
}
```

Доступ к множеству всех ребер графа

- Для доступа к ребрам графа – свойство E.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.E" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "rating": 92,
      "_id": 0,
      "_type": "edge",
      "_outV": 1,
      "_inV": 0,
      "_label": "REPORTED_ON"
    },
    {
      "style": "ice wine",
      "_id": 1,
      "_type": "edge",
      "_outV": 0,
      "_inV": 2,
      "_label": "grape_type"
    }
  ],
  "success": true
}
```

Доступ к конкретной вершине графа

- Получить конкретную вершину можно, передав номер узла методу v.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.v(0)" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "name": "Prancing Wolf Ice Wine 2007",
      "_id": 0,
      "_type": "vertex"
    }
  ],
  "success": true
}
```

Свойства конкретной вершины графа

- Для перечисления свойств вершины графа используйте метод `map()`.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.v(0).map()" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "name": "Prancing Wolf Ice Wine 2007"
    }
  ],
  "success": true
}
```

Фильтрация вершин по значению

- Чтобы получить конкретный узел (вершину) по имени, можно воспользоваться синтаксисом фильтра {...}, который в Groovy называется замыканием.
- Весь код внутри скобок определяет функцию; если она возвращает true, то программа посещает соответствующую вершину.
- Переменная it внутри замыкания представляет текущий объект.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.V.filter{it.name=='riesling'}" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "name": "riesling",
      "_id": 2,
      "_type": "vertex"
    }
  ],
  "success": true
}
```

Получение исходящих из вершины ребер

- Имея вершину, можно получить исходящие из нее ребра, вызвав метод `outE()`.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.V.filter{it.name=='Wine Expert Monthly'}.outE()" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "rating": 92,
      "_id": 0,
      "_type": "edge",
      "_outV": 1,
      "_inV": 0,
      "_label": "REPORTED_ON"
    }
  ],
  "success": true
}
```

- Для получения входящих ребер служит метод `inE()`, а для получения одновременно входящих и исходящих – метод `bothE()`.

Получение вершин, в которые ведут исходящие ребра

- Зная исходящие ребра, можно пройти в соседние вершины, вызвав метод `inV`, то есть получить вершины, в которые ведут ребра.
- Ребро `reported_on`, исходящее из узла `Wine Expert`, ведет в вершину `Prancing Wolf Ice Wine 2007`, поэтому вызов `outE.inV` ее и вернет.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.V.filter{it.name=='Wine Expert Monthly'}.outE().inV()" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "name": "Prancing Wolf Ice Wine 2007",
      "_id": 0,
      "_type": "vertex"
    }
  ],
  "success": true
}
```

Получение вершин, из которых ведут ребра в заданное множество вершин

- Обратная операция реализуется выражением `inE.outV`.
- В Gremlin есть сокращенная нотация для таких выражений. Выражение `out` – это сокращенная запись для `outE.inV`, а выражение `in` – для `inE.outV`.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.V.filter{it.name=='Wine Expert Monthly'}.out.name" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    "Prancing Wolf Ice Wine 2007"
  ],
  "success": true
}
```

Добавление новой вершины (узла)

- В одном винодельческом хозяйстве могут производиться разные вина, и, если планируется хранить несколько вин, то необходимо добавить виноградник в качестве связующей вершины, из которой ведет ребро в вершину Prancing Wolf.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= pwolf = g.addVertex([name : 'Prancing Wolf Winery'])" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "name": "Prancing Wolf Winery",
      "_id": 8,
      "_type": "vertex"
    }
  ],
  "success": true
}
```

Добавление новой связи (ребра)

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= pwolf = g.addVertex([name : 'Prancing Wolf Winery'])" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "name": "Prancing Wolf Winery",
      "_id": 8,
      "_type": "vertex"
    }
  ],
  "success": true
}
```

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= g.addEdge(g.v(8), g.v(0), 'produced')" http://localhost:7474/tp/gremlin/execute | jq
{
  "results": [
    {
      "_id": 2,
      "_type": "edge",
      "_outV": 8,
      "_inV": 0,
      "_label": "produced"
    }
  ],
  "success": true
}
```

Выполнение нескольких выражений в одном скрипте

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=pwolf = g.addVertex([name: 'Prancing Wolf Winery']);  
g.addEdge(pwolf, g.v(0), 'produced');" http://localhost:7474/tp/gremlin/execute |jq  
{  
  "results": [  
    {  
      "_id": 2,  
      "_type": "edge",  
      "_outV": 3,  
      "_inV": 0,  
      "_label": "produced"  
    }  
  ],  
  "success": true  
}
```

Добавление новых вершин

- Теперь можно добавить еще два рислинга: Kabinett и Spatlese.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=kabinett = g.addVertex([name: 'Prancing Wolf Kabinett 2002']);  
g.addEdge(g.v(3), kabinett, 'produced');" http://localhost:7474/tp/gremlin/execute |jq  
{  
  "results": [  
    {  
      "_id": 3,  
      "_type": "edge",  
      "_outV": 3,  
      "_inV": 4,  
      "_label": "produced"  
    }  
  ],  
  "success": true  
}
```

Добавление новых вершин

- Теперь можно добавить еще два рислинга: Kabinett и Spatlese.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=spatlese = g.addVertex([name: 'Prancing Wolf Spatlese 2007']);  
g.addEdge(g.v(3), spatlese, 'produced');" http://localhost:7474/tp/gremlin/execute |jq  
{  
  "results": [  
    {  
      "_id": 4,  
      "_type": "edge",  
      "_outV": 3,  
      "_inV": 5,  
      "_label": "produced"  
    }  
  ],  
  "success": true  
}
```

Добавление новых ребер

- Пополните этот небольшой граф, добавив ребра, ведущие из вершины riesling к новым вершинам.
- Установите переменную riesling, отфильтровав вершину riesling.

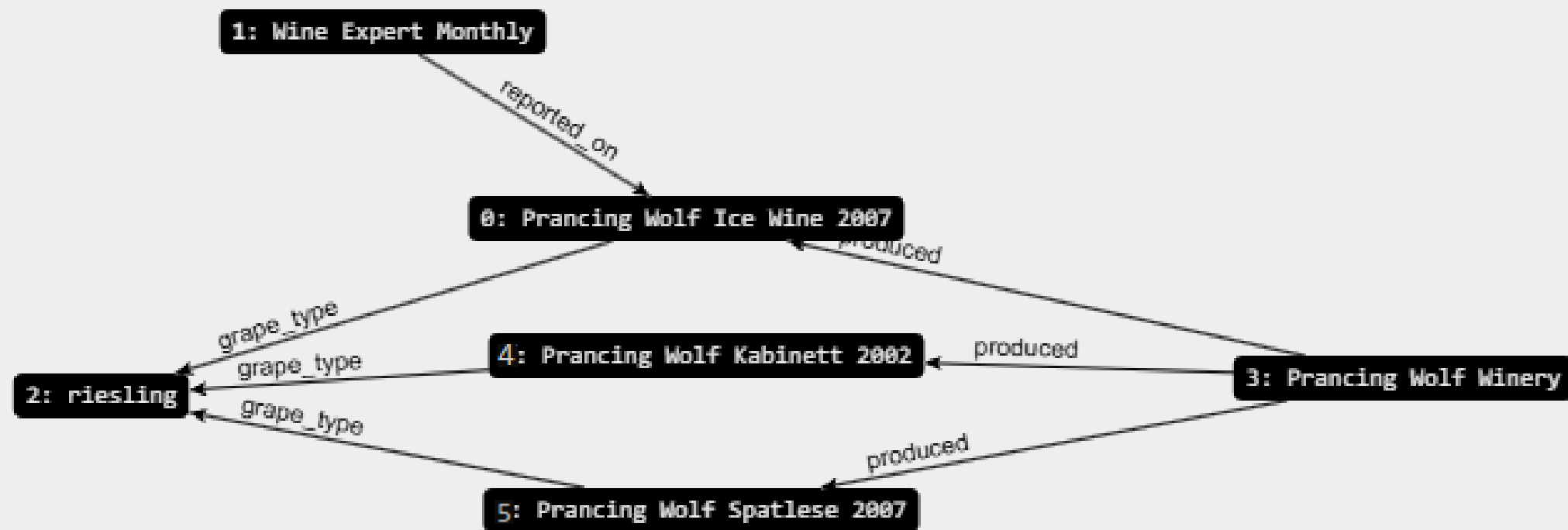
```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=riesling = g.V.filter{it.name=='riesling'}.next();  
g.addEdge([style:'kabinett'], g.v(4), riesling, 'grape_type')" http://localhost:7474/tp/gremlin/execute |  
jq  
{  
  "results": [  
    {  
      "_id": 5,  
      "_type": "edge",  
      "_outV": 4,  
      "_inV": 2,  
      "_label": "grape_type"  
    }  
  ],  
  "success": true  
}
```

Добавление новых ребер

- Вершину Spatlese можно соединить ребром с riesling аналогично.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=riesling = g.V.filter{it.name=='riesling'}.next();  
g.addEdge([style:'spatlese'], g.v(5), riesling, 'grape_type')" http://localhost:7474/tp/gremlin/execute |  
jq  
{  
  "results": [  
    {  
      "_id": 6,  
      "_type": "edge",  
      "_outV": 5,  
      "_inV": 2,  
      "_label": "grape_type"  
    }  
  ],  
  "success": true  
}
```

Граф узлов после добавления данных с помощью Gremlin



Графовые базы данных: Neo4j

§4. Конвейеры



Суть конвейеров

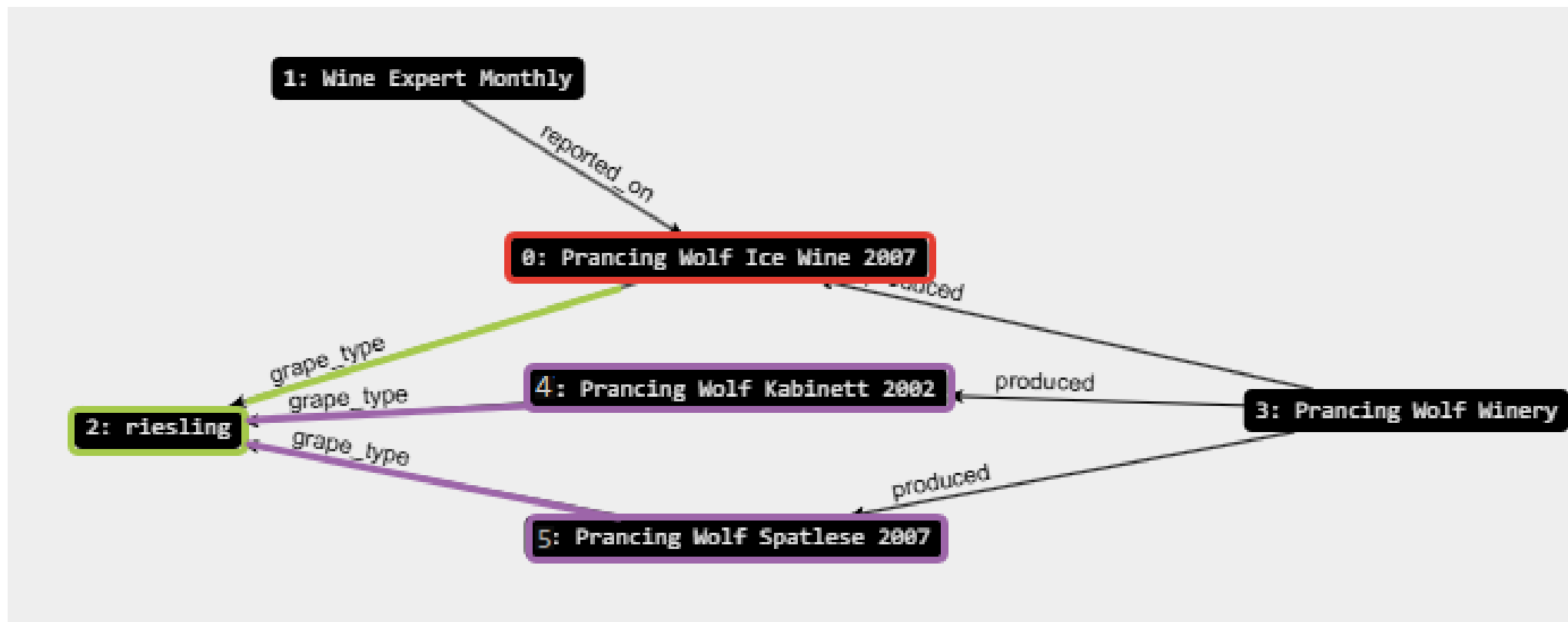
- Операции в Gremlin – последовательность каналов (pipe).
- Каждый канал на входе принимает коллекцию, а на выходе возвращает другую коллекцию.



- Элементами могут быть вершины, ребра и значения свойств.
- Конвейер – последовательность каналов.
- Gremlin – язык для построения каналов.

Исследование идеи каналов с помощью графа узлов

- Необходимо найти вино, похожие на данное, - то есть изготовленные из того же сорта винограда.



Исследование идеи каналов с помощью графа узлов

```
script=ice_wine = g.v(0); ice_wine.out('grape_type').in('grape_type').filter{ !it.equals(ice_wine) }
```

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=ice_wine = g.v(0); ice_wine.out('grape_type').in('grape_type').filter{ !it.equals(ice_wine) }" http://localhost:7474/tp/gremlin/execute |jq
```

```
{
  "results": [
    {
      "name": "Prancing Wolf Spatlese 2007",
      "_id": 5,
      "_type": "vertex"
    },
    {
      "name": "Prancing Wolf Kabinett 2002",
      "_id": 4,
      "_type": "vertex"
    }
  ],
  "success": true
}
```

Конвейер и вершина

- Чтобы получить коллекцию, содержащую только одну указанную вершину, можно отфильтровать ее от списка всех узлов.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= g.V.filter{it.name=='riesling'}.next()" http://localhost:7474/tp/gremlin/execute |jq
{
  "results": [
    {
      "name": "riesling",
      "_id": 2,
      "_type": "vertex"
    }
  ],
  "success": true
}
```



Краткая характеристика Cypher

- Cypher – поддерживаемый Neo4j язык запросов к графам.
- Основан на сопоставлении с образцами и имеет SQL-подобный синтаксис.
- Перейдите на начальный веб-интерфейс Neo4j и введите следующий эквивалент запроса «похожие вина» в строку запросов:

```
1 START ice_wine=node(0)
2 MATCH (ice_wine) -[:grape_type]-> () <-[:grape_type]- (similar)
3 RETURN similar
```

similar

name Prancing Wolf Spatlese 2007

name Prancing Wolf Kabinett 2002

Свойство фильтра class

- Фильтр возвращает объект GremlinGroovyPipeline (конвейер).

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= g.V.filter{it.name=='riesling'}.class" http://localhost:7474/tp/gremlin/execute |jq
{
  "results": [
    "class com.tinkerpop.gremlin.groovy.GremlinGroovyPipeline"
  ],
  "success": true
}
```

- Если сравнить это с классом следующего узла, полученного из конвейера. Это будет не что иное – Neo4j2Vertex (вершина).

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= g.V.filter{it.name=='riesling'}.next().class" http://localhost:7474/tp/gremlin/execute |jq
{
  "results": [
    "class com.tinkerpop.blueprints.impls.neo4j2.Neo4j2Vertex"
  ],
  "success": true
}
```

Графовые базы данных: Neo4j

§5. Бессхемная социальная сеть



Добавление социального аспекта

#1

- Предположим, требуется добавить трех человек, знакомых между собой, у каждого из которых есть любимые вина. Алисе нравится ледяное вино.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= alice = g.addVertex([name:'Alice']);  
ice_wine = g.V.filter{it.name=='Prancing Wolf Ice Wine 2007'}.next(); g.addEdge(alice, ice_w  
ine, 'likes')" http://localhost:7474/tp/gremlin/execute |jq  
{  
  "results": [  
    {  
      "_id": 7,  
      "_type": "edge",  
      "_outV": 6,  
      "_inV": 0,  
      "_label": "likes"  
    }  
  ],  
  "success": true  
}
```

Добавление социального аспекта #2

- Тому нравится Кабинет и ледяное вино и доверяет всему, что пишет Wine Expert Monthly.

```
tom = g.addVertex([name:'Tom']);  
ice_wine = g.V.filter(it.name=='Prancing Wolf Ice Wine 2007').next();  
kabinett = g.V.filter(it.name=='Prancing Wolf Kabinett 2002').next();  
g.addEdge(tom, kabinett, 'likes'); g.addEdge(tom, ice_wine, 'likes');  
g.addEdge(tom, g.V.filter(it.name=='Wine Expert Monthly').next(), 'trusts')
```

```
{  
  "results": [  
    {  
      "_id": 10,  
      "_type": "edge",  
      "_outV": 7,  
      "_inV": 1,  
      "_label": "trusts"  
    }  
  ],  
  "success": true  
}
```

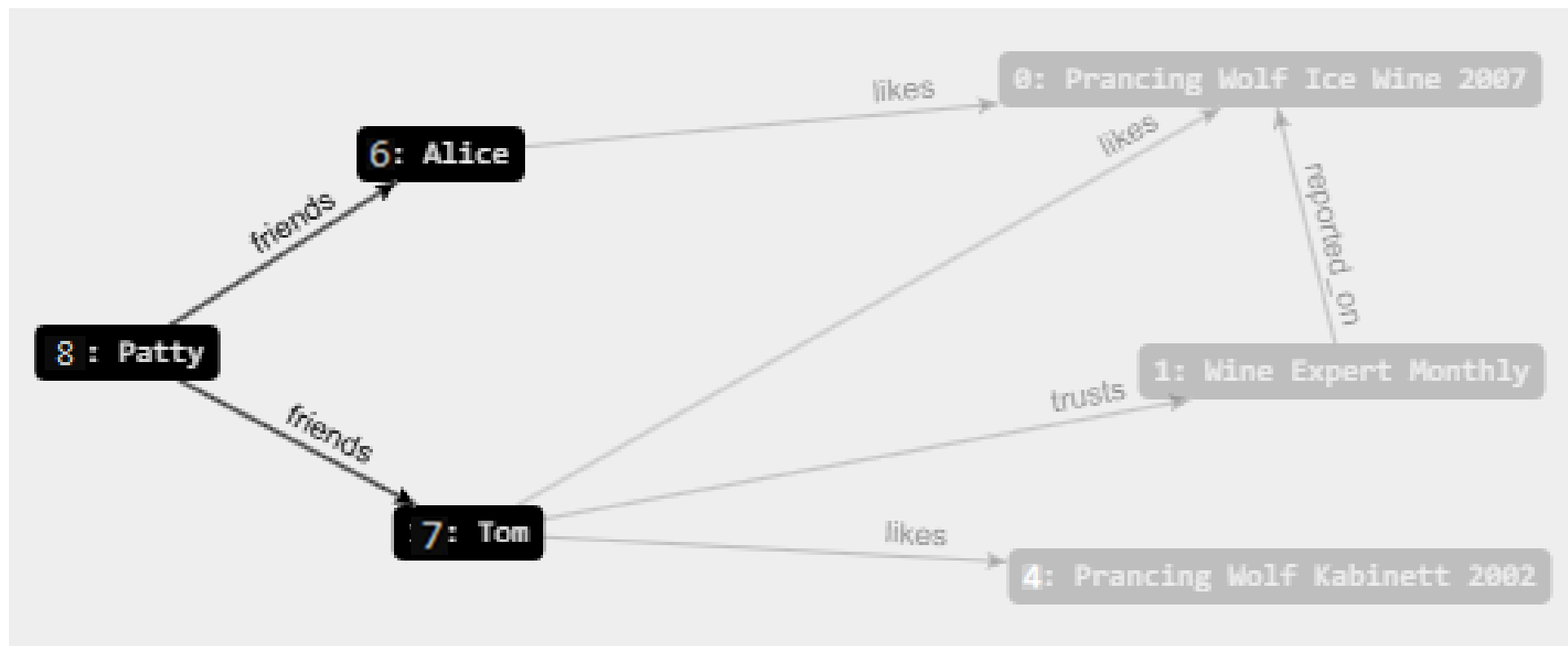
Добавление социального аспекта #3

- Пэтти дружит с Томом и Алисой; предпочтений в винах нет.

```
patty = g.addVertex([name:'Patty']);  
tom = g.V.filter{it.name=='Tom'}.next();  
alice = g.V.filter{it.name=='Alice'}.next();  
g.addEdge(patty, tom, 'friends');  
g.addEdge(patty, alice, 'friends')
```

```
{  
  "results": [  
    {  
      "_id": 12,  
      "_type": "edge",  
      "_outV": 8,  
      "_inV": 6,  
      "_label": "friends"  
    }  
  ],  
  "success": true  
}
```

Фрагмент графа узлов после добавления социального аспекта



Графовые базы данных: Neo4j

§6. Дополнительные шаги обработки каналов



Методы bothE и bothV

- Методы inE, outE, inV, outV – шаги трансформации, позволяющие извлекать входящие и исходящие ребра и вершины.
- Методы bothE и bothV следуют вдоль ребра вне зависимости от того, является оно входящим или исходящим.
- Следующий запрос находит Алису и всех ее друзей:

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= alice =  
g.V.filter{it.name=='Alice'}.next(); alice.bothE('friends').bothV  
.name" http://localhost:7474/tp/gremlin/execute |jq  
{  
  "results": [  
    "Patty",  
    "Alice"  
  ],  
  "success": true  
}
```

Фильтр except()

- Если Алиса нас не интересует, то можно включить в конвейер фильтр except(), передав ему список ненужных узлов:

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= alice =  
g.V.filter{it.name=='Alice'}.next(); alice.bothE('friends').bothV  
.except([alice]).name" http://localhost:7474/tp/gremlin/execute |  
jq  
{  
  "results": [  
    "Patty"  
  ],  
  "success": true  
}
```

Другие вариации фильтров

- Фильтр `retain()` посещает только те узлы, которые ему переданы.
- Можно отфильтровать вершину с помощью кода, проверяющего, что данная вершина не совпадает с `alice`.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= alice =  
g.V.filter{it.name=='Alice'}.next(); alice.bothE('friends').bothV  
.filter{!it.equals(alice)}.name" http://localhost:7474/tp/gremlin  
/execute |jq  
{  
  "results": [  
    "Patty"  
  ],  
  "success": true  
}
```

Поиск вширь: друзья друзей

- Чтобы определить с кем дружит Алиса, необходимо повторить шаги:

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= alice =  
g.V.filter{it.name=='Alice'}.next(); alice.bothE('friends').bothV  
.except([alice]).bothE('friends').bothV.except([alice]).name" htt  
p://localhost:7474/tp/gremlin/execute |jq  
{  
  "results": [  
    "Patty",  
    "Patty",  
    "Tom"  
  ],  
  "success": true  
}
```

Метод loop()

- Метод loop() повторяет предыдущие шаги, пока заданное замыкание остается равным true.
- loop(3) – повторение трех предшествующих шагов, отсчитывая назад точки перед вызовом метода loop: except – первый шаг, bothV – второй шаг, bothE – третий шаг.
- После каждой итерации loop() вызывает переданное ему замыкание.

```
alice.bothE('friends').bothV.except([alice]).loop(3) {  
    it.loops <= 2  
}.name
```

```
{  
  "results": [  
    "Patty",  
    "Patty",  
    "Tom"  
  ],  
  "success": true  
}
```

Фильтрация дубликатов

- Фильтр `dedup()` убирает дублирующиеся значения.

```
alice.bothE('friends').bothV.except([alice]).loop(3) {  
    it.loops <= 2  
}.dedup().name
```

```
{  
  "results": [  
    "Patty",  
    "Tom"  
  ],  
  "success": true  
}
```

Демонстрация пути

- Чтобы понять, как были получены эти значения, можно проследовать по пути друг->друг, воспользовавшись преобразованием path():

```
alice.bothE('friends').bothV().except([alice]).loop(3) {  
    it.loops <= 2  
}.dedup().name().path()
```

```
{  
  "results": [  
    [  
      {  
        "name": "Alice",  
        "_id": 6,  
        "_type": "vertex"  
      },  
      {  
        "_id": 12,  
        "_type": "edge",  
        "_outV": 8,  
        "_inV": 6,  
        "_label": "friends"  
      }  
    ]  
  ],  
}
```

```
{  
  {  
    "name": "Patty",  
    "_id": 8,  
    "_type": "vertex"  
  },  
  {  
    "_id": 12,  
    "_type": "edge",  
    "_outV": 8,  
    "_inV": 6,  
    "_label": "friends"  
  },  
}
```

```
{  
  {  
    "name": "Patty",  
    "_id": 8,  
    "_type": "vertex"  
  },  
  "Patty"  
],  
}
```

Обход в обратном направлении

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= alice =  
g.V.filter{it.name=='Alice'}.next(); alice.outE.inV.back(2).name"  
http://localhost:7474/tp/gremlin/execute |jq  
{  
  "results": [  
    "Alice"  
  ],  
  "success": true  
}
```

Группировка элементов и их подсчет

- Шаг `groupCount()/countBy` обходит узлы и подсчитывает повторяющиеся значения, сохраняя их в ассоциативном массиве.
- Обход всех вершин графа и подсчет, сколько раз встречаются различные значения свойства `name`:

```
name_map = [:];  
g.V.name.groupCount(name_map);  
name_map
```

```
g.V.name.countBy{it}
```

- Подсчитайте, сколько марок вина любит каждый зарегистрированный в системе человек:

```
g.V.outE('likes').outV.name.countBy{it}
```

Графовые базы данных: Neo4j

§7. Переход на Groovy



НОВЫЕ ВОЗМОЖНОСТИ

- В Groovy имеет функция распределения, которая называется `collect()`, и функция редуцирования `inject()`.
- Пусть требуется подсчитать, сколько вин еще не получили оценку.
 - Сначала необходимо построить список значений `true/false`, показывающих, оценено вино или нет.
 - Затем этот список подается на вход редуктора, который подсчитывает количество `true` и `false`.

Фаза распределения

- `g.V.in('grape_type')`: возвращает вершины, в которые входит ребро типа `grape_type`.
- В замыкании `collect` смотрит, входит ли в текущую вершину ребро типа `reported_on`.
- Вызов метода `toList()` преобразует конвейер в список, который можно проверить на пустоту.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.V.in('grape_type').collect
{ !it.inE('reported_on').toList().isEmpty() }" http://localhost:7474/tp/gremlin/execu
te |jq
{
  "results": [
    false,
    false,
    true
  ],
  "success": true
}
```

Фаза редуцирования

- Аргумент 0 в inject(0) служит для инициализации count нулем перед первым обращением.
- Оператор «стрелка» отделяет входные аргументы замыкания от его тела.
- В теле замыкания либо возвращается текущее значение счетчика count, либо увеличивается на единицу.
- На выходе получаем количество значений false в списке.

```
rated_list.inject(0){ count, is Rated ->
  if (is Rated) {
    count
  } else {
    count + 1
  }
}
```

```
{
  "results": [
    2
  ],
  "success": true
}
```

Различные комбинации обходов и трансформаций графа #1

- Пусть требуется найти все пары друзей.
- Необходимо найти все ребра типа friends, а затем вывести имена людей на обоих концах ребра.
- Здесь замыкание transform возвращает литеральный массив ([...]) из двух элементов: входящей и исходящей вершин ребра friends.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script=g.V.outE('friends').transform  
[[it.outV.name.next(), it.inV.name.next()]]" http://localhost:7474/tp/gremlin/execute  
[]  
{  
  "results": [  
    [  
      "Patty",  
      "Alice"  
    ],  
    [  
      "Patty",  
      "Tom"  
    ]  
  ],  
  "success": true  
}
```

Различные комбинации обходов и трансформаций графа #2

- Чтобы найти всех людей и вина, которые они любят, необходимо преобразовать каждого найденного человека (вершина на любом конце ребра типа friends) в список из двух элементов: имя человека и список его любимых вин.

```
g.V.both('friends').dedup.transform{
    [ it.name, it.out('likes').name.toList() ]
}
```

```
{
  "results": [
    [
      "Patty",
      []
    ],
    [
      "Alice",
      [
        "Prancing Wolf Ice Wine 2007"
      ]
    ],
    [
      "Tom",
      [
        "Prancing Wolf Ice Wine 2007",
        "Prancing Wolf Kabinett 2002"
      ]
    ]
  ],
  "success": true
}
```

Графовые базы данных: Neo4j

§8. Предметно-ориентированные шаги



Предметно-ориентированный язык

- «Из какой вершины, в которую входит ребро типа `grape_type`, исходит ребро, ведущее в данную вершину, описывающую вино?».
 - «Каковы сортовые характеристики этого вина?»
-
- Gremlin ориентирован на предметную область: запрос к графовым БД.
 - Gremlin предоставляет новые шаги, семантически связанные с хранящимися в графе данными.

Создание шага

- Создайте отдельный текстовый файл. Переименуйте в `varietal.gremlin` и сохраните в папку `scripts`, расположенную в `Program Files`.
- В начале сообщаем ядру Gremlin, что собираемся определить шаг с именем `varietal`.
- Во второй строке: новый шаг необходимо присоединить к классам `Vertex` и `Pipe`.
- В последней строке: замыкание, содержащее исполняемый шагом код.
- `_()` – это текущий объект конвейера.

```
Gremlin.defineStep( 'varietal',  
    [Vertex, Pipe],  
    {_().out('grape_type').dedup}  
)
```

ВЫЗОВ ШАГА

- Новый шаг вызывается, как любой другой.
- Необходимо добавить дополнительную команду для взаимодействия с внешними скриптами.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= g.V.filter{it.name=='Prancing Wolf  
Ice Wine 2007'}.varietal.name" --data-urlencode "load=varietal" http://localhost:7474/tp/gremlin/execute |jq  
{  
  "results": [  
    "riesling"  
  ],  
  "success": true  
}
```

- Запрос вернул сорт винограда, из которого изготовлено ледяное вино.

Создание шага #2

- Создадим еще один шаг: получение любимых вин всех друзей.
- Как и раньше, необходимо дать новому шагу имя – friendsuggest – и сохранить в виде отдельного файла.

```
Gremlin.defineStep( 'friendsuggest',  
    [Vertex, Pipe],  
    {  
        _().both('friends').  
        except([it]).out('likes').dedup  
    }  
)
```

Вызов шага #2

- И теперь новый шаг можно включить в конвейер как обычно.

```
C:\WINDOWS\system32>curl -s -G --data-urlencode "script= g.V.filter{it.name=='Patty'}.friendsuggest.name"
--data-urlencode "load=friendsuggest" http://localhost:7474/tp/gremlin/execute |jq
{
  "results": [
    "Prancing Wolf Ice Wine 2007",
    "Prancing Wolf Kabinett 2002"
  ],
  "success": true
}
```

- Использование имеющихся в Groovy средств метапрограммирования для создания новых шагов. — мощное средство разработки предметно-ориентированных языков.

Графовые базы данных: Neo4j

§9. Обновление, удаление и стирание



Обновление свойств

- Добавьте оценку Алисы к вину Prancing Wolf Ice Wine 2007 года, приписав ей вес.

```
e=g.V.filter{it.name=='Alice'}.outE('likes').next();  
e.weight = 95;  
e.save
```

Удаление значений

```
e=g.V.filter(it.name=='Alice').outE('likes').next();  
e.removeProperty('weight');  
e.save
```

Стирание графа

- У объекта, представляющего граф, есть функции для удаления вершин и ребер: `removeVertex` и `removeEdge` соответственно.
- Чтобы уничтожить граф, можно удалить все вершины и ребра:

```
g.V.each{ g.removeVertex(it) }  
g.E.each{ g.removeEdge(it) }
```

- Проверить, что ничего не осталось, можно, вызвав методы `g.V` и `g.E`.
- Того же результата можно достичь с помощью метода `clear()` (`g.clear()`)
- Если была запущена консольная оболочка Gremlin (вне веб-интерфейса), то рекомендуется корректно закрывать соединение с графом методом `shutdown()`.

Самостоятельная работа



Задача #1

1. Создайте новый граф, представляющий вашу семью (или семью известной личности).
2. Присвойте свойства узлам и связям.
3. ГБД заархивируйте и загрузите в вашу папку на Яндекс.Диск

Графовые базы данных: Neo4j

